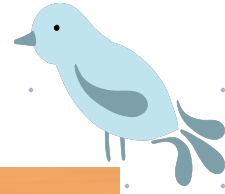


TREES I



Today's content

- Trees Intro
- Naming convention
- Trees traversal
- Iterative inorder
- Construct tree with `in[]` & `post[]`

✓ Mock Interview

✓ DSA Full syllabus contest (5 question) → 3 attempts



Mock Interview (2 questions)

Fixed Mock
credits

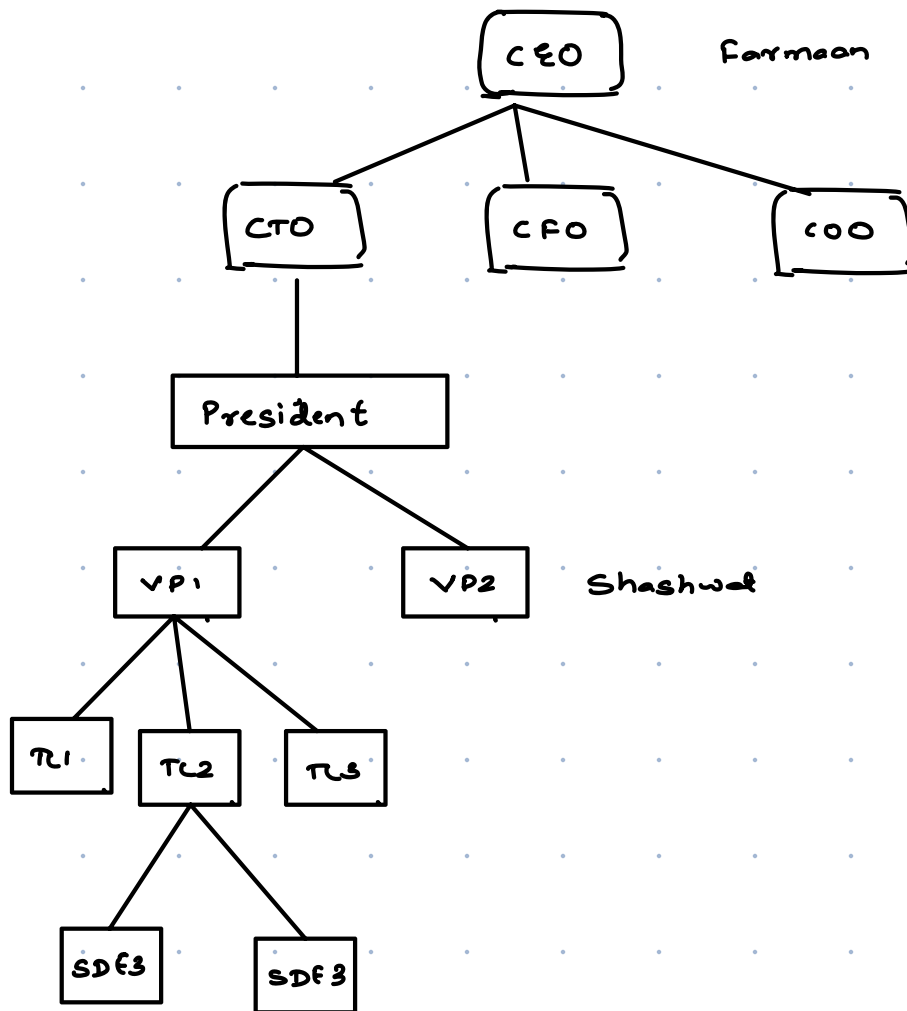
Floater
Mock credits
without expiry

DSA clear Full contest → +30 days
clear Mock Interview

Linear Data structure

01. Array
02. Linkedlist
03. Arraylist
04. Stack & Queue

Trees → Data structure which stores hierarchical data.

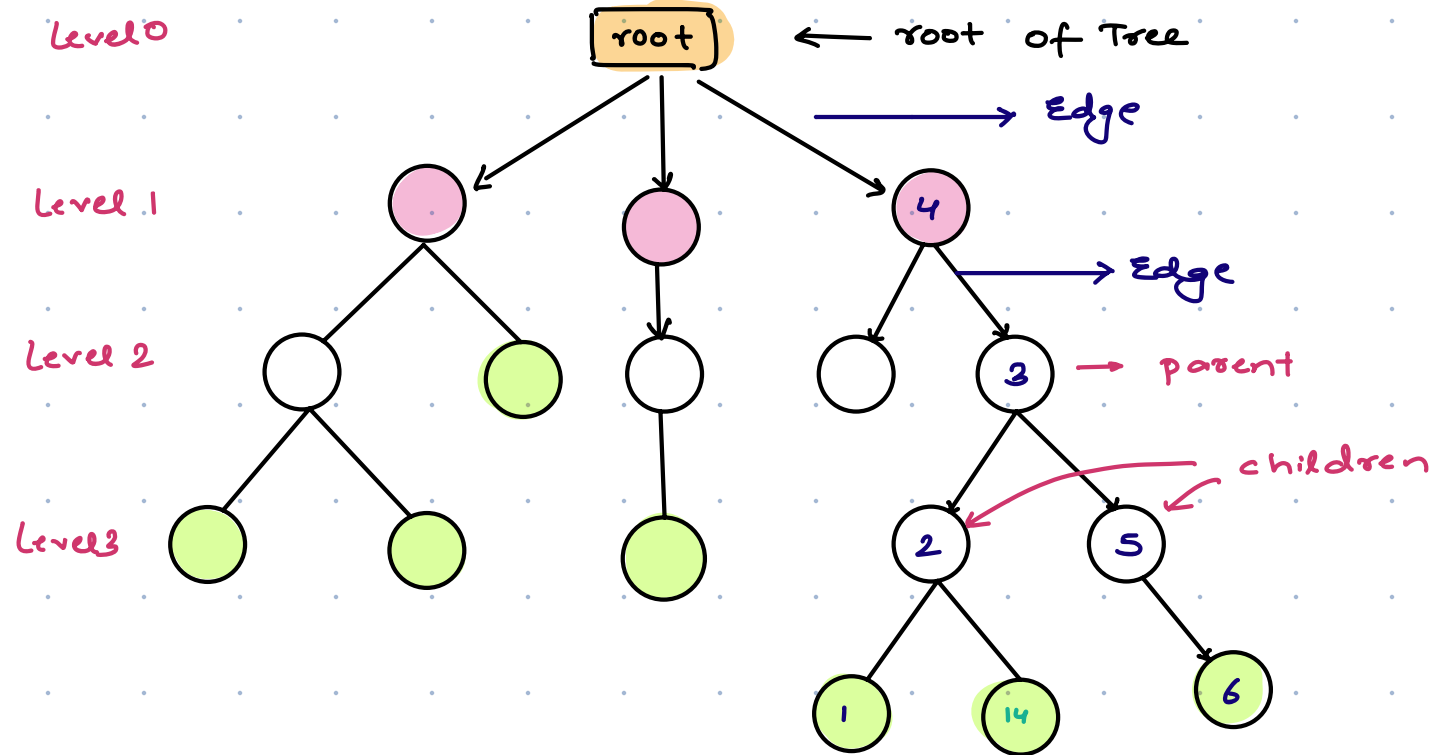


02. Family tree

03. Folder structure

03. Git commits

Naming convention of Trees



● → sibling nodes → share the same parent.

● → leaf nodes → Nodes with 0 child

* Every node will have one parent except the root node.

Total no. of Nodes = N

No. of Edges = $N - 1$

Ancestor $\rightarrow$ All nodes from parent to root node in upward direction are going to be ancestors

$$\text{Ancestor}(1) = 2, 3, 4, \text{root}$$

Descendants $\rightarrow$ All nodes from child to leaf nodes along the path are descendants

$$\text{Descendant}(3) = 1, 2, 4, 5, 6$$

* Height of a node $\rightarrow$ Maximum distance (in terms of edges) between the node & its descendant leaf nodes

* Height of tree = Height of root node $\Rightarrow \underline{4}$

* Depth of a Node $\rightarrow$ No. of edges between the given node & the root node $\text{depth}(\text{root}) = 0$

Depth of tree = Depth of farthest leaf node

Height of Tree = Depth of tree

* Subtree $\Rightarrow$ A portion of a tree

$\rightarrow$ Node along with its descendant, which are part of a tree.

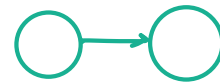
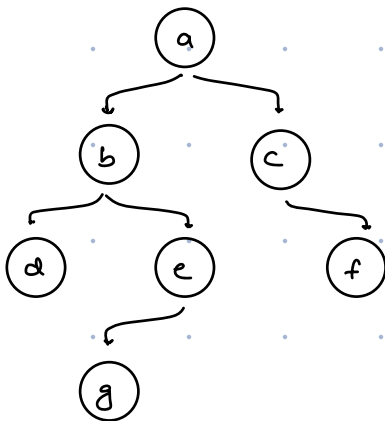
Quiz 1 = Can a leaf node be a subtree? $\Rightarrow$ Yes

Quiz 2 = Do all nodes have a parent node? $\Rightarrow$ No, root node doesn't have parent node.

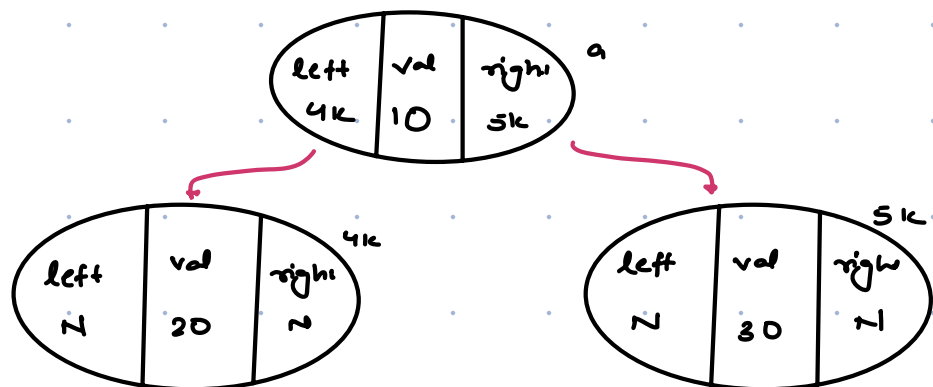
Quiz 3 = Height of leaf node = 0

* Binary Tree $\rightarrow$ Tree in which node can have atmost 2 children

$\{0, 1, 2\}$



```
class Node{  
    int val;  
    Node left;  
    Node right;  
}
```

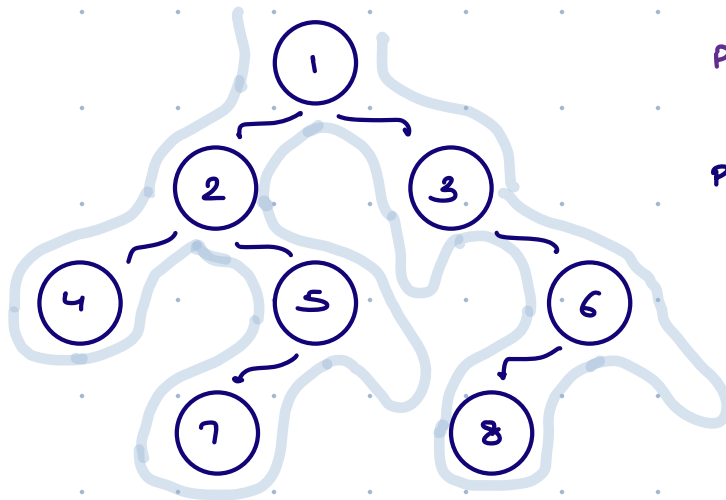


a.left = 4k

a.right = 5k

* Binary Tree Traversal

01. Preorder { Node LST RST }
02. Inorder { LST Node RST }
03. Post order { LST RST Node }



Pre = { 1 2 4 5 7 3 6 8 }

post = { 4 7 5 2 8 6 3 1 }

10:23 pm → 10:33 pm

Preorder Traversal

{ N L R } - For every node

```
void preorder (Node root) {
```

```
    if (root == null) return;
```

```
    print (root.val);
```

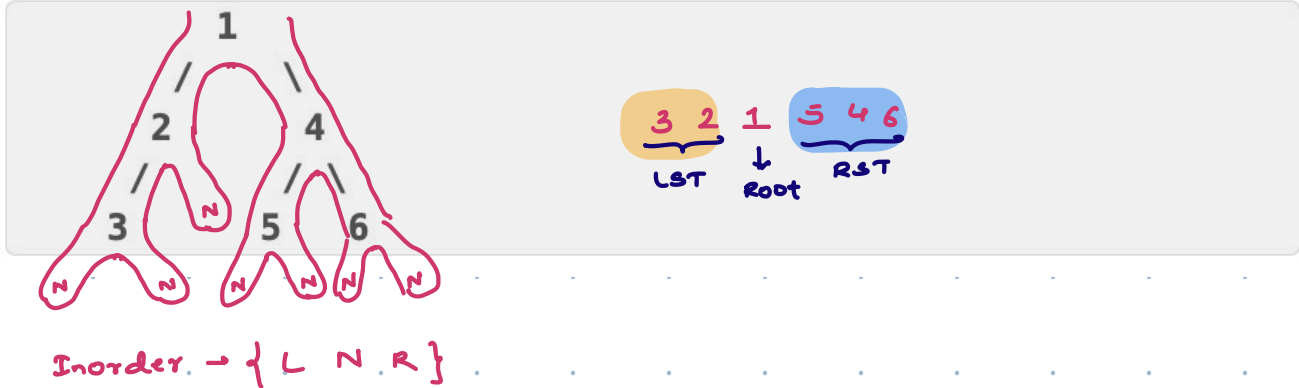
```
    preorder (root.left)
```

```
    preorder (root.right)
```

Tc : $O(n)$

sc : $O(\text{ht of tree})$

What is the inorder traversal sequence of the below tree?



```
void inorder (Node root){  
    if (root == null) return;  
    inorder (root.left)  
    print (root.val);  
    inorder (root.right)  
}
```

Tc : $O(n)$
sc : $O(\text{ht of tree})$

Scenario

In an e-commerce platform like **Flipkart**, when orders are placed, they need to be processed efficiently based on their time of placing the order. **Flipkart's** approach is to assign **OrderIDs** in increasing order for orders placed based on their times.

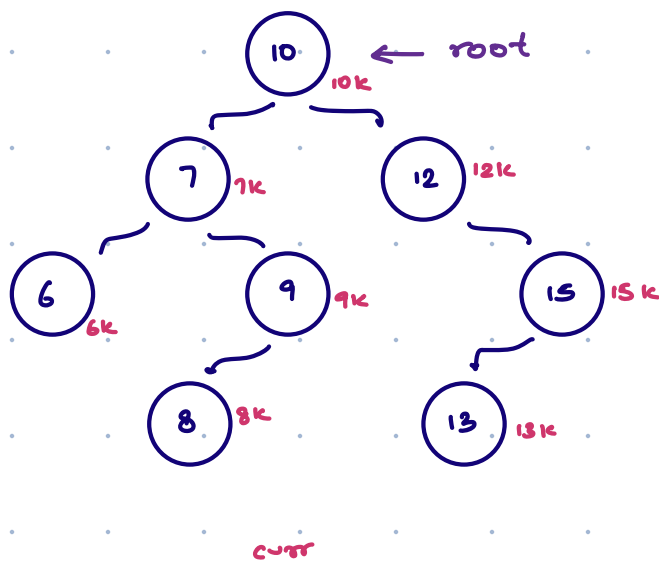
Problem Statement

You are given a Binary Tree, each order can be represented as a node in a binary tree, and having a value denoting the **OrderID** of the order.

- The **left child** represents the order placed before it and,
- The **right child** represents the order placed after it.

Q Need to tell the order in which all these booking will get processed

{ Inorder Traversal }



L N R

8k
9k
6k
7k
10k

while (st.size() > 0 || curr != null) {

while (curr != null)

st.push(curr);

curr = curr.left

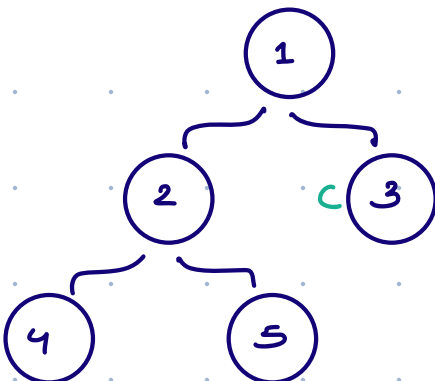
} LST as far as you can

curr = st.pop();

print(curr.val);

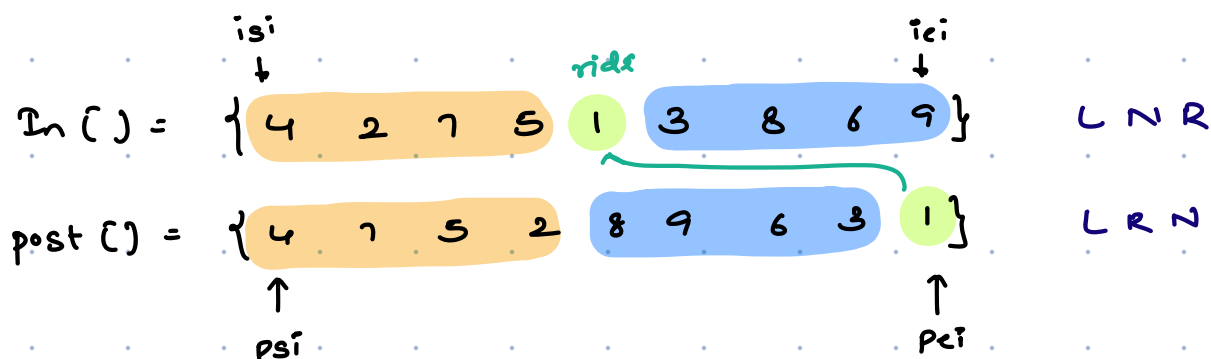
curr = curr.right;

4 2 5 1



5k
4k
2k
1k

Q Construct the binary tree using inorder() & postorder()

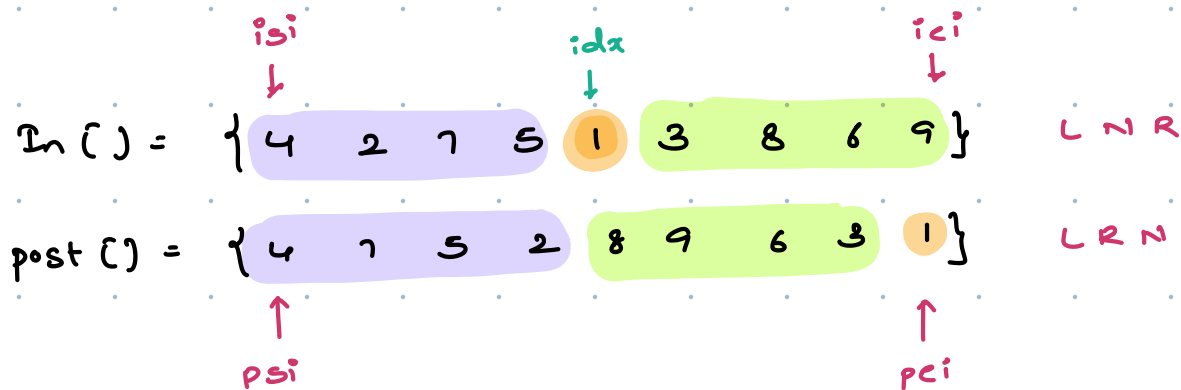


* Find the root node = $post[pci]$

* Find this root node in inorder array = $ridx$

→ Get the count of nodes in LST

→ Get the count of nodes in RST



$In() = LST = \{isi, idx - 1\}$

$RST = \{idx + 1, ici\}$

$$\text{count} = a = \text{isi}$$

$$b = \text{idx} - 1$$

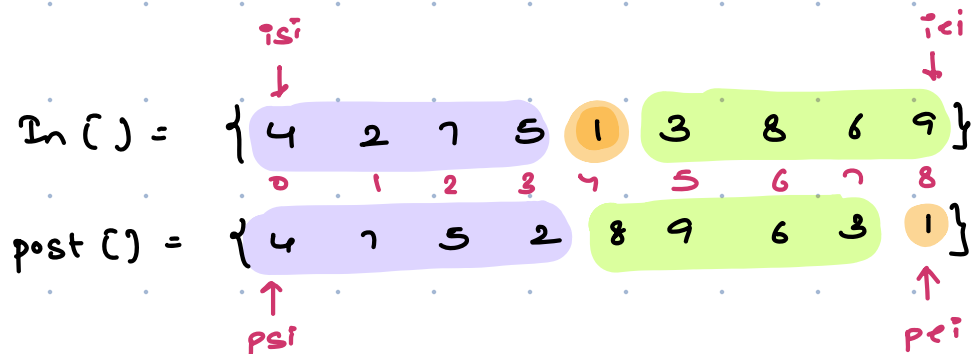
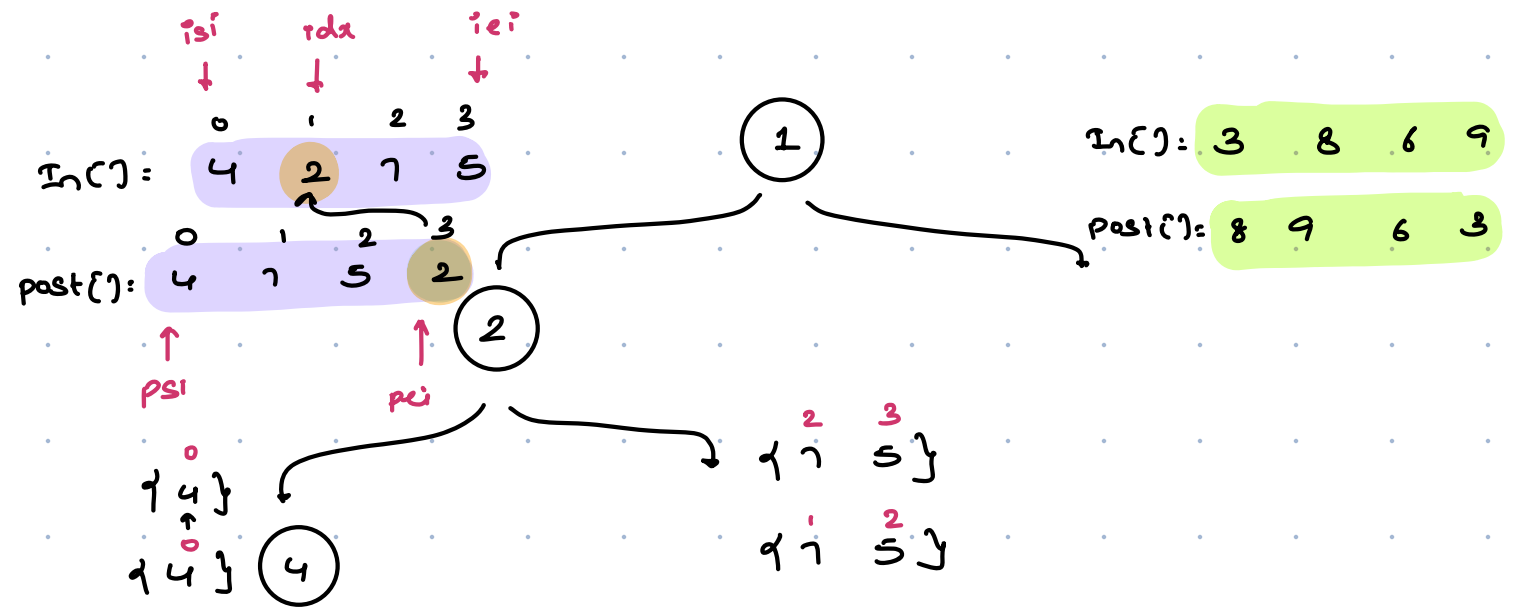
$$\text{count of ele} = b - a + 1$$

$$= \text{idx} - 1 - \text{isi} + 1$$

$$\text{count} = \text{idx} - \text{isi}$$

$$\text{post}[] = \text{LST} = \{ \text{psi}, \text{psi} + (\text{idx} - \text{isi}) - 1 \}$$

$$\text{RST} = \{ \text{psi} + (\text{idx} - \text{isi}), \text{pei} - 1 \}$$



Node construct (in [], post [], 0, n-1, 0, n-1) {

if (isi > iei) return null;

Node root = new Node (post [pei])

int idx = -1

for (i = isi ; i ≤ iei ; i++) {

if (in[i] == post[pei]) {

idx = i;

break;

}

root.left = construct (in, post, isi, idx-1, psi, psi+(idx-isi)-1)

root.right = construct (in, post, idx+1, iei, psi+(idx-isi), pei-1);

return root;